Ben-Gurion University of the Negev

Faculty of Engineering Science

Dept. of Electrical and Computer Engineering

Image processing

“Simon Says” course project

Assigned by:

Nitay Peled

Amit Lasky

Dor Buchnik

Adam Sidi

1.Abstract:

This project presents a Simon Says game developed in Python, integrated with a Rabbids-themed interface. The game uses basic image processing techniques, including agglomerative clustering, mean-shift segmentation, and filtering, to analyze player movements. Through real-time image processing, the system detects user gestures to determine if they correctly follow the game's instructions. Developed as a final project for an image processing course, this work explores the application of fundamental segmentation and filtering techniques in an interactive setting.

2.Introduction:

Description:

Our project is a real-time interactive game that utilizes hand and gesture recognition for two players, inspired by "Simon Says" with a unique twist using computer vision techniques. In the game, an Non-Player Character (NPC) provides directional commands such as "up," "down," "right," or "left," while another NPC gives the opposite instructions. Players must correctly position their hands based on the first NPC's command while Doing the opposite direction of the second. Each correct move earns a point, and the player with the highest score at the end wins. The game is developed in Python, incorporating image processing algorithms for gesture and skin detection to accurately track player movements in real time.

3.Project goal:

The project aimed to develop a real-time skin segmentation system with live face tracking, ensuring it doesn't mistake other objects for human skin. The game screen is split into two halves, assigning each player to one side to avoid interference. The system maintains accuracy despite changes in lighting, skin tone, and movement.

4.Expected result:

We expect to achieve real-time hand tracking using skin segmentation and the Mean Shift motion tracking algorithm. The system should accurately detect and follow hand movements while avoiding false detections caused by other objects in the frame. This ensures precise tracking and responsiveness during gameplay.

System description:

The system operates using a the main computer camera, which captures video footage of the two players standing in front of it. The players can see themselves on the computer screen, which is divided into two sections, each assigned to one player. At the beginning of the game, the system calibrates the players' hand positions by prompting them to place their hands inside designated green rectangles. This calibration step ensures accurate hand tracking throughout the game. Once calibration is complete, the game begins, and the interactive NPCs (non playable character), which are connected to the program, provide real-time instructions and responses based on the players' actions.

For this project, we assume that there will only be two players, and the system is designed to function exclusively with them. It does not account for the presence of a third person or any additional individuals entering the camera's field of view. The system will strictly track and recognize only the two calibrated players, ensuring that any external interference or intrusions within the camera's sight do not affect the gameplay.

Challenges:

One major challenge is object intrusion. Since the game is played in a library with various objects in the field of view, and no white screen is used as a background, the system must effectively differentiate between human skin and surrounding objects. This requires reliable segmentation techniques to avoid false detections.

Initially, we attempted to rely solely on hand detection without head detection. However, due to difficulties in distinguishing between 'idle' and 'down' hand positions, we needed to incorporate head tracking. While detecting the head was relatively straightforward, maintaining accurate tracking over time introduced additional challenges.

Another challenge we faced involved computing and real-time connectivity to the game, given hardware limitations such as camera frame rate analysis and the execution of real-time code on the computer. Additionally, we were restricted from using more complex algorithms or larger codebases, as they negatively impacted connectivity, slowed down detection, and ultimately reduced the game's fun and responsiveness.

5.Algorithms description

1- Skin segmentation :

```
def skin_segmentation(image):
```

For hands recognition, we implemented a skin detection algorithm using agglomerative clustering. The users begin by placing their hands within a calibration square, where the program segment it by *agglomerative clustering of the H-S (V is not used because of the variation in the brightness) color space. Then mask the square to contain only the estimated skin pixels - based on the dominant labels identified in the segmentation. From this, a histogram of skin color distribution is generated.

The mask and histogram is created with the function **:**

```
hist = get_hs_histogram(image, None)
```

*The `get_hs_histogram` function is largely based on the concepts covered in Lecture 2 of the course and has been highly beneficial in addressing the challenge of varying skin tones among players.

*Agglomerative clustering Explanation: A build in python function taking the number of clusters and the distance thresholds between each clusters, the output is a cluster arrays with labels assigned to each data. The clusters are computed as :

$$Let\ X = \{x_1, x_2, \dots \dots x_n\}\ the\ data\ set\ then:$$

$$at\ first\ we\ initialize\ each\ point\ as\ a\ cluster$$

$$C_1 = \{x_1\}, , , C_n = \{x_n\}$$

$$then\ D_{i,j} = d(C_i, C_j)\ we\ compute\ the\ distance\ between\ each\ cluster$$

$$we\ find\ the\ pair\ \{i^*, j^*\} = argmin\{D_{i,j}\}$$

$$we\ assigned\ C_{new} = C_{i^*} \cup\ C_{j^*}$$

$$The\ process\ continue\ as\ the\ number\ of\ clusters\ we\ assigned\ converge.$$

The Agglomerative Clustering Algorithm was discussed in Lecture 12 of the course and has been extremely helpful in reducing false hand detection and improving tracking.

From the obtained clusters, we generate a histogram of the calibrated image pixels based on their assigned labels. This histogram serves as a probability distribution function, estimating the likelihood of each pixel in the frame belonging to the detected skin labels. The reference histogram is created in the calibration process with the function:

```
def extract_hands_hist(roi_img1, roi_img2):
```

we will use later the reference histogram for the masking process

The back projection and mask is the computed by :

```
def apply_backprojection(target_img, ref_hist, thresh=50)
```

the function use the probability function to compute the mask the P.F is calculated as seen below:

$$P(H,S) = \frac{\left(Histogram\ count(H,S)\right)}{\sum Histogram\ count(H',S')}$$

$$where\ the\ binary\ mask\ will\ be\ computed:$$

$$M(x,y) = \begin{cases} 1, & if\ P(H,S) > T \\ 0, & else \end{cases}$$

$$T\ is\ a\ defined\ treshold\ in\ the\ line:$$

```
_, thresholded = cv2.threshold(back_proj, thresh, 255, cv2.THRESH_BINARY)
```

After that the function apply morphological operations to remove noise in the image using kernels and cv2 library.

To improve the masking we also applied large contours filters using:

```
def filter_large_and_far_contours(skin_mask, min_area=1000, max_dist=100):
```

$$let\ C = \{c_1, \dots, c_n\}\ be\ the\ set\ of\ countours\ detected$$

$$the\ countour\ will\ be\ kept\ if:$$

$$Area(c_i) > \min area\ \text{ and }\ \text{distance}(c_i, largest\ countour) < \max dist$$

*The distance is the Euclidian distance.

Once the live frames comes one by one, the program mask each frame by back projection of the sampled hands histograms.

2-Head tracking

For head tracking, we implemented a *Mean-Shift algorithm, which continuously computes the mean position of the head in each frame, dynamically updating its location in real-time.

*The Mean Shift Algorithm was also discussed in Lecture 12 of the course and played a crucial role in addressing head tracking challenges.

Mean shift function:

The head position is calibrated using the region of interest initialized by the user in the beginning of the game

```
head_window_1 = roi_head_1
head_window_2 = tuple(np.array(roi_head_2) + shift)
heads_windows = [head_window_1, head_window_2]
```

the centroid mean of the head is then computed at each frame by:

$$C(x, y) = \sum (x_i, y_i) P(x_i, y_i)$$

Where $P(x_i, y_i)$ is the probability function used from the back projection. We draw a rectangle at the new position centroid at each frame:

```
_, new_window = cv2.meanShift(masked_backproj_1, window, term_crit)
new_window = avoid_shrinking(new_window)
new_window = reinit_window(new_window, masked_backproj_1)
draw_rect(frame_with_mask1, new_window, "red")
heads_windows[i] = new_window
```

We assume the head moves only small distances per frame. If tracking is lost, the mean is reinitialized and recalculated.

3-Position clasification:

Finally, the program classifies hand positions by segmenting the screen into predefined regions: up, down, left, right, and idle, detecting where the hand is located and categorizing its movement accordingly with the function:

```
def detect_movement(self, mask, min_area, regions, player, down=False):
```

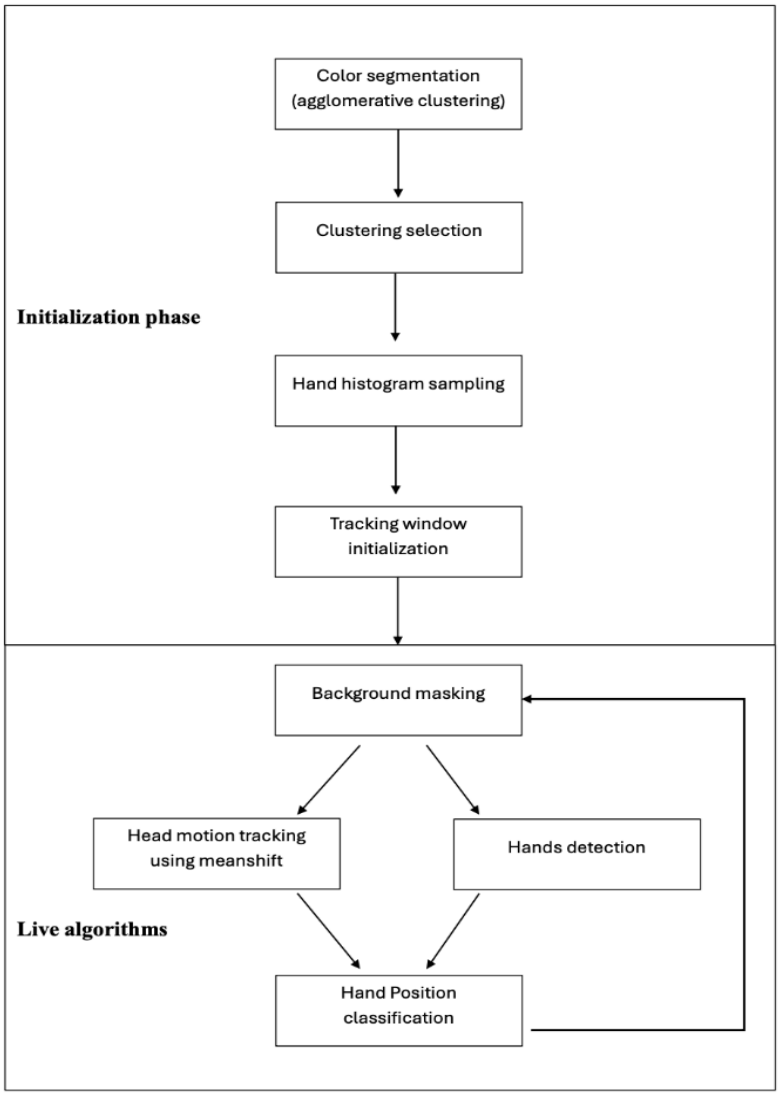
The Detect Movement function and class primarily serve as a state machine that tracks the player's movement.
For left, right, and up movements, we define three rectangular regions in the user's frame. Each frame, we count the number of skin pixels in these regions and classify movement as:

$$\left\{\begin{matrix} \text{Left}\,, & if\ A_{left} = \max\left(A_{left}, A_{right}, A_{up}\right) \\ \text{right}\,, & if\ A_{right} = \max\left(A_{left}, A_{right}, A_{up}\right) \\ \text{up}\,, & if\ A_{up} = \max\left(A_{left}, A_{right}, A_{up}\right) \end{matrix}\right\}$$

Where A is the skin pixel area detected in the decision region. If the A below a treshold A_{min} the position is classified by "idle"

For the down movement we classified it by the head tracking ,If the head centroid move below 35% of the frame height the position is then set to "down".

Here is given a flow diagram of the algorithm:



$figure\ 1: Algorithm\ flow$

4-Game connectivity:

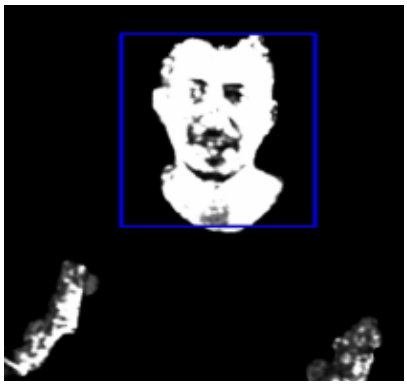
The program is written as a state machine for two players where the states are :

“up” , “down” ,”right” ,”left”, “idle” . for each one.

The game is connected to the game “Rabbids game of legends” via ubisoft. Each state in the state machine is connected to a keyboard Arrow key for player one and a switch via an external emulated vga controller for player two.

The game inputs are the detected keys pressed that are triggered by the state machine and classified by the above’s function.

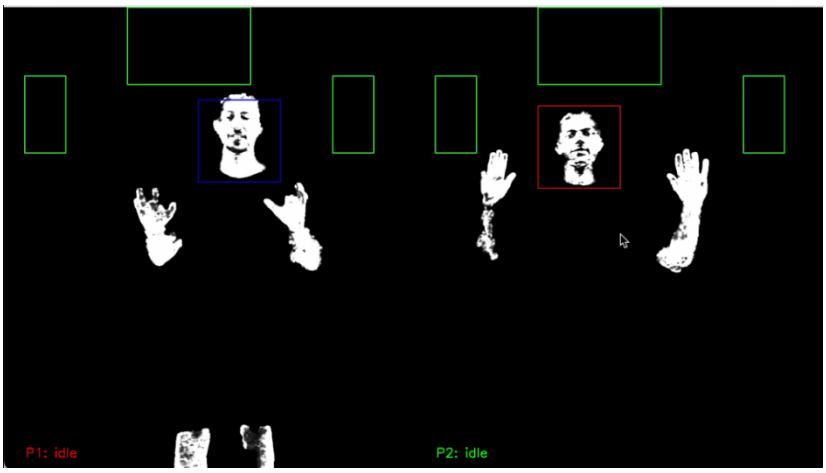
6.Preliminary Results:



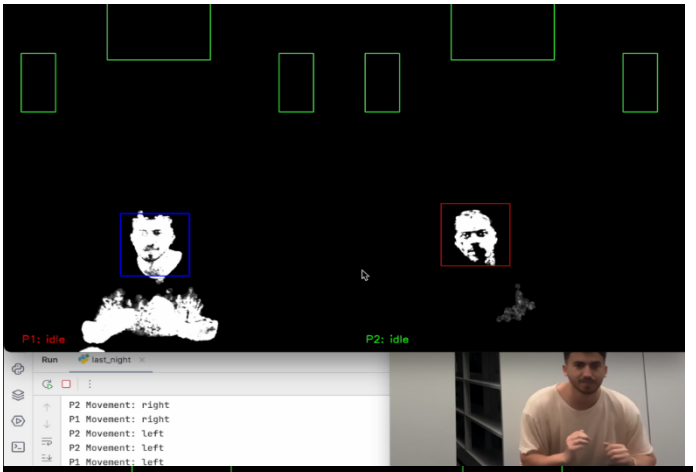
1. Mean shift is used to identify the head, which is characterized by its relatively large size and slow movement. The algorithm calculates the average position of the pixels to determine the location of the bounding box accurately.

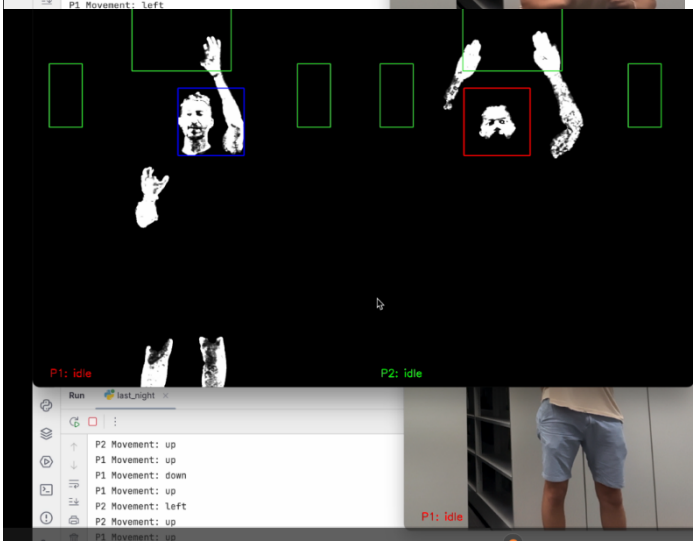
2. Hand detection – we use a skin detection algorithm based on agglomerative clustering. Users start by placing their hands in a calibration square, where the program segments the skin in the **H-S color space**. The square is then masked to retain only the estimated skin pixels, identified by the dominant cluster labels.

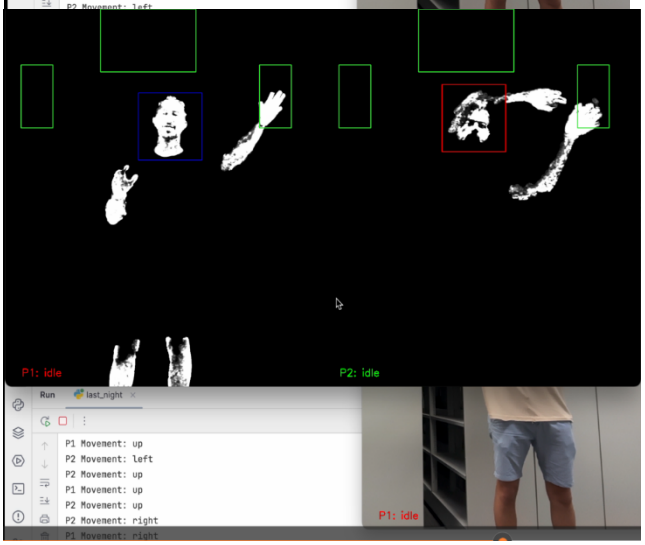
The results:



3. <u>Classified hand positions</u> - by segmenting the screen into predefined regions (up, down, left, right, idle) and detecting the hand's location and movement accordingly.





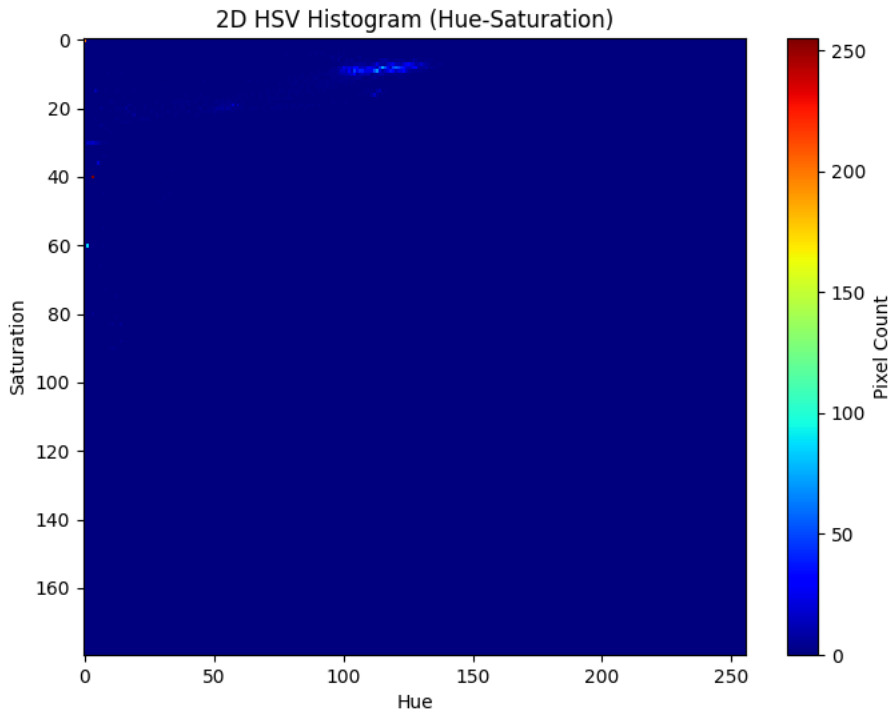


7. Analysis of Agglomerative Clustering implementation

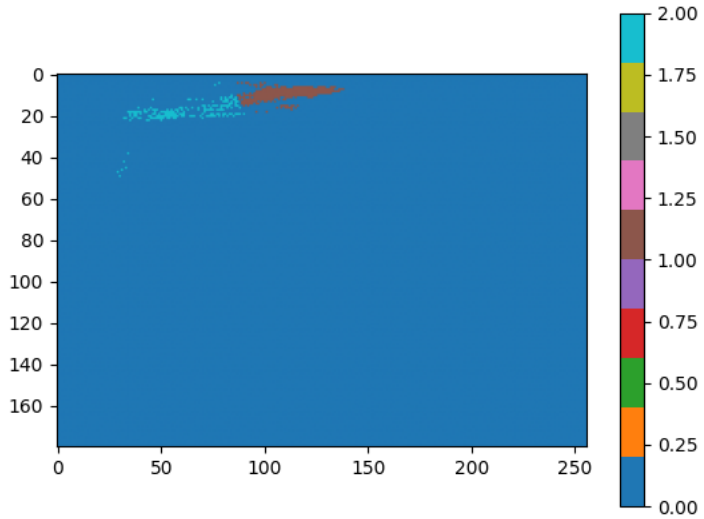
We chose to analyze the segmentation of the hand image at the calibration stage which is achieved by agglomerative clustering implementation, using the Precision and Recall quantitative measurements .

The first analysis is performed on this hand image that taken during the calibration stage:

There are other objectives in the image and there are parts in the image with colors similar to the hand colors. The program first calculates the Hue and Saturation histogram:



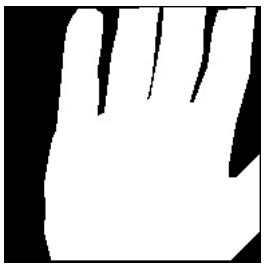
It can be noticed that the histogram is sparse and there are few clusters at the top and the top left. Then, Agglomerative Clustering is used to cluster the histogram:



The algorithm stops when there is sufficient distance between the clusters, so it is not known in advance how many clusters will be at the end of the algorithm (in this example there were 3). For that reason, we chose to select the k (in the final code we choose k=2) most common labels that appear at a predetermined circle at the center of the ROI (the hand image) and mask the image by them:

It can be seen that the mask is mostly comprised of the hand (indicating high precision). However, there are a few areas, such as the index finger, where the mask failed to detect the hand, reducing recall. Additionally, there is a blob on the left that is incorrect, which lowers precision.

Now, for calculating precision and recall, manual mask was drawn:

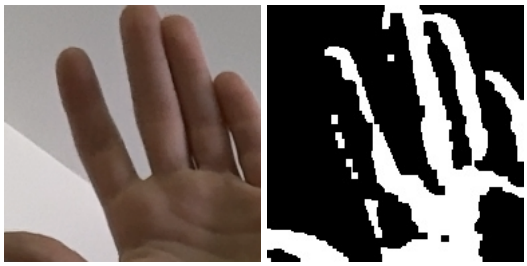


And the measurements calculated as taught in class:

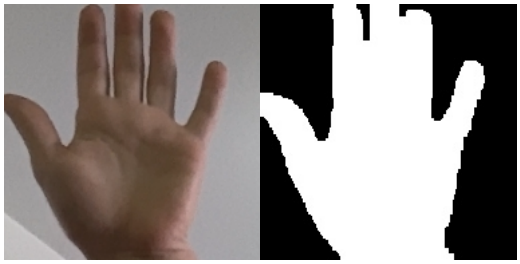
```
Precision: 0.9619
Recall: 0.7984
```

As expected, we got high precision. It is important to mention that for our needs the precision measurement is more important than recall. High precision means that we captured mostly the hand color information and not background color information. That will lead to eliminating noise in the filtering of the upcoming frames. On the other hand, recall is important for getting the whole color information of the hand – so higher recall will lead to more skin color detection in the upcoming frames. However, with this recall we got sufficient results. It was more important for us to eliminate noise than get more skin parts.

A few more test images are analyzed shortly in the same manner:



```
Precision: 0.9504
Recall: 0.4854
```



```
Precision: 0.9504
Recall: 0.9709
```

```
Precision: 0.9445
Recall: 0.2829
```

With all these calibration masks, even the ones with low recall, we still got good skin detection in the next frames because of their high precision.

Summary:

This project focuses on the creation of a "Simon Says" interactive game with a Rabbids-themed interface, developed in Python using fundamental image processing techniques. The system integrates algorithms such as agglomerative clustering for skin segmentation, mean-shift tracking for head detection, and gesture recognition to track real-time player movements. The results demonstrate that these algorithms provided robust and reliable performance, showcasing their adaptability even in dynamic environments. A significant portion of this project relied on algorithms and concepts discussed during the course, such as skin segmentation and tracking techniques, enabling the team to deepen their understanding by applying these methods in real-time scenarios. However, the overall system was constrained by hardware limitations, such as the camera and computing resources required for processing large codes and analyses efficiently. Additionally, the fast-paced nature of the game posed challenges in maintaining gameplay fluidity, which could be improved by using better hardware to enhance the fun and interactive experience.

Conclusions:

- We gained valuable experience by experimenting with various approaches and algorithms, aiming to make the game both interactive and enjoyable. This process enhanced our understanding of course-taught techniques, allowing us to implement and improve them effectively.
- Overall, the game was highly enjoyable and interactive. Despite the challenges faced, we learned a great deal, both technically and practically.
- To further enhance interactivity, incorporate more complex algorithms, or add additional players, stronger hardware—especially a more advanced camera and CPU—will be required.
- Most of the development was done outside the library where the project was presented. While this made the work more challenging due to the complex environment, it allowed us to refine the algorithms to perform effectively in difficult conditions. With adjustments for specific environments, such as lighting, performance could be even better.
- This project highlighted the importance of leveraging foundational image processing techniques to solve practical problems in interactive systems.
- The reliance on a heavy, pre-existing game imposed restrictions in terms of both image processing and fast analysis. Developing a custom-built game could optimize real-time dynamics and system responsiveness in future iterations.

Bibliography:

Most of the referenced documents are explained in the report and marked with a red *, but additional links and tools used for assistance and information are listed below:

- Course Website - Presentations, and Recordings [https://moodle.bgu.ac.il/moodle/course/view.php?id=56058]
- Stack Overflow & ChatGPT – Mostly for syntax and code assistance [https://stackoverflow.com/questions]
- Wikipedia, GeeksforGeeks, and other websites – Additional resources for algorithm information [https://www.geeksforgeeks.org, https://www.wikipedia.org/].